

Millisecond Data Serving from Lance

UB-pool tiered cache · Sampler prefetch · Cost-aware placement

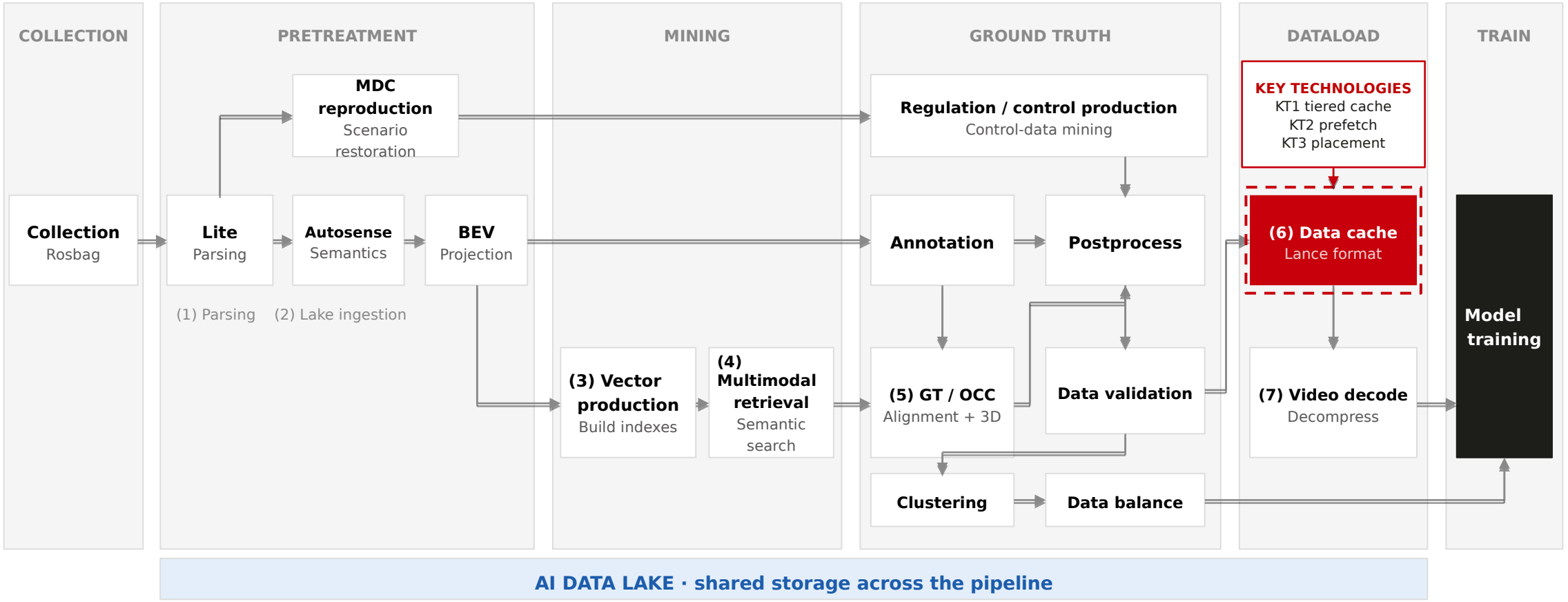
Research proposal v1 · 11 November 2026

Vector Index Optimization

HUAWEI

We speed up ADS stage (6): serving the data lake

[Company context] Redrawn from ADS-PLAN slide 1; target is (6) Data cache in Lance format: a 4 GB clip from OBS at ms level.

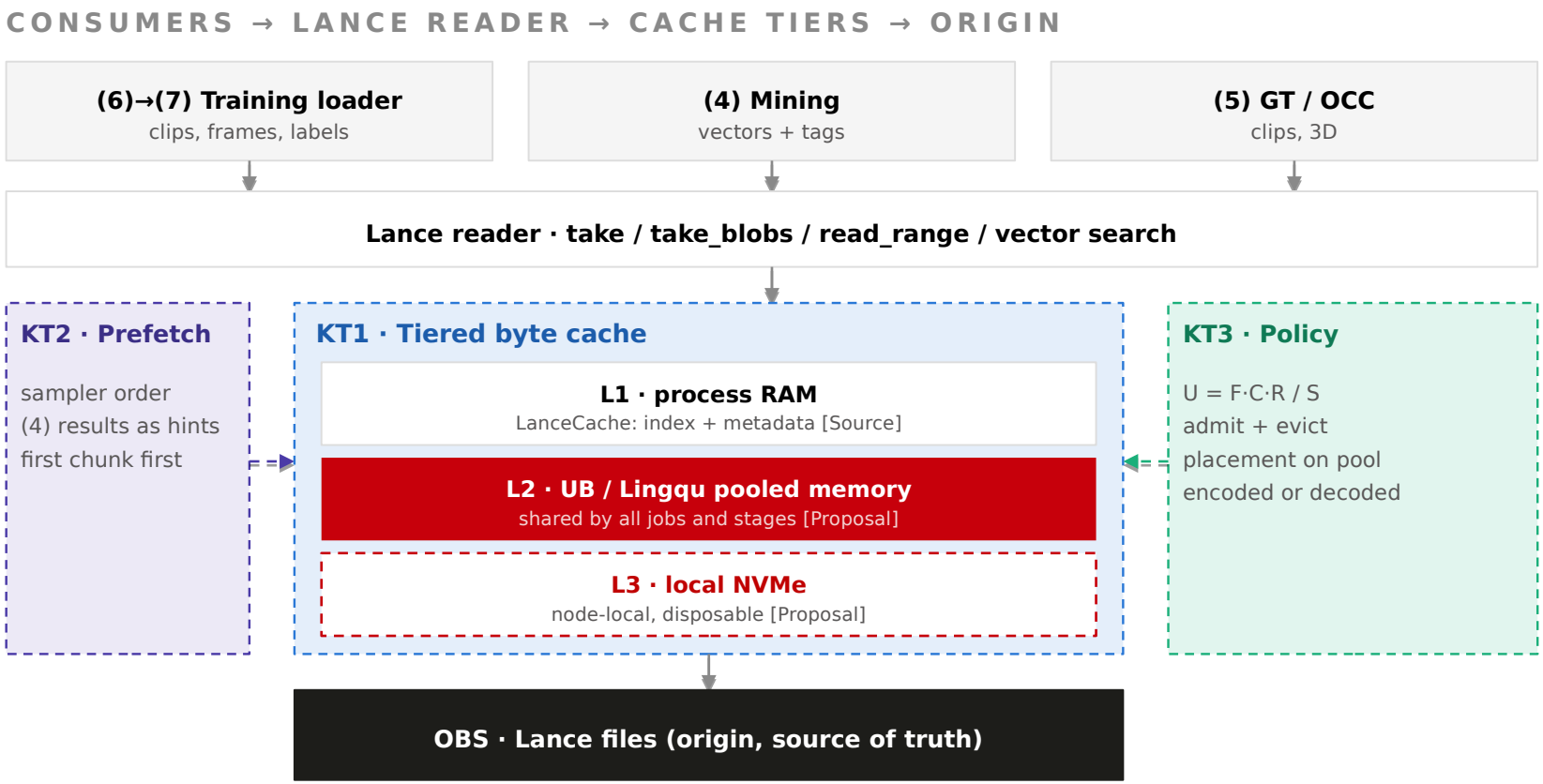


Training waits on the lake: a 4 GB clip from OBS must reach ms level. All three key technologies sit in this box.

– Huawei Confidential –

Three key technologies in the Lance read path

Consumers read objects, vectors and tags through Lance; each key technology (KT) at its integration point · gaps [Source] · dashed = proposed



Every tier is a disposable copy of immutable Lance files: a miss or a lost node falls back to OBS

STATUS · 11 NOV 2026

KT1 tiered cache

- Gap verified: no data-byte cache
- Hooks exist: object-store wrapper,
- CacheTier::Local (no backend)

KT2 prefetch

- Training order known ahead [Assumption]
- Blob range reads exist [Source]

KT3 policy

- Baselines in Lance: S3-FIFO, TinyLFU
- OriginLatencyClass: RQ switch only

KT1 adds the tiers; KT2 fills them ahead of need; KT3 decides what stays. Each ships separately.

– Huawei Confidential –

"ms" needs a close hit and a partial read

[Model] transfer time at aggregate bandwidth B · [Source] what Lance caches today · UB / NVMe / OBS figures on ADS nodes are TBD

Model · transfer time = bytes ÷ aggregate bandwidth

Bandwidth B	Whole 4 GB clip	64 MiB window
1 GB/s	4,000 ms	62 ms
10 GB/s	400 ms	**6.2 ms**
50 GB/s	80 ms	**1.2 ms**
100 GB/s	40 ms	**0.6 ms**

Transfer only; first-byte latency, decode and queueing come on top. A whole clip stays above 40 ms even at 100 GB/s.

Two levers

Read less
the range a step needs: 64× fewer bytes than the clip

Read closer
from L2 UB pool, not OBS: first-byte latency TBD

What Lance caches today [Source]

Data	Cached?	Where / how
IVF partitions (RQ codes)	yes	index cache 6 GiB, S3-FIFO
Manifests, file + page metadata	yes	metadata cache 1 GiB
Deletion masks	yes	metadata cache, per version
Column + blob bytes (video, lidar)	**no**	**512 KiB read-ahead per BlobFile**
Shared or persistent tier	**no**	**CacheTier::Local, no backend**
Spill store	scratch	index build only, not a cache

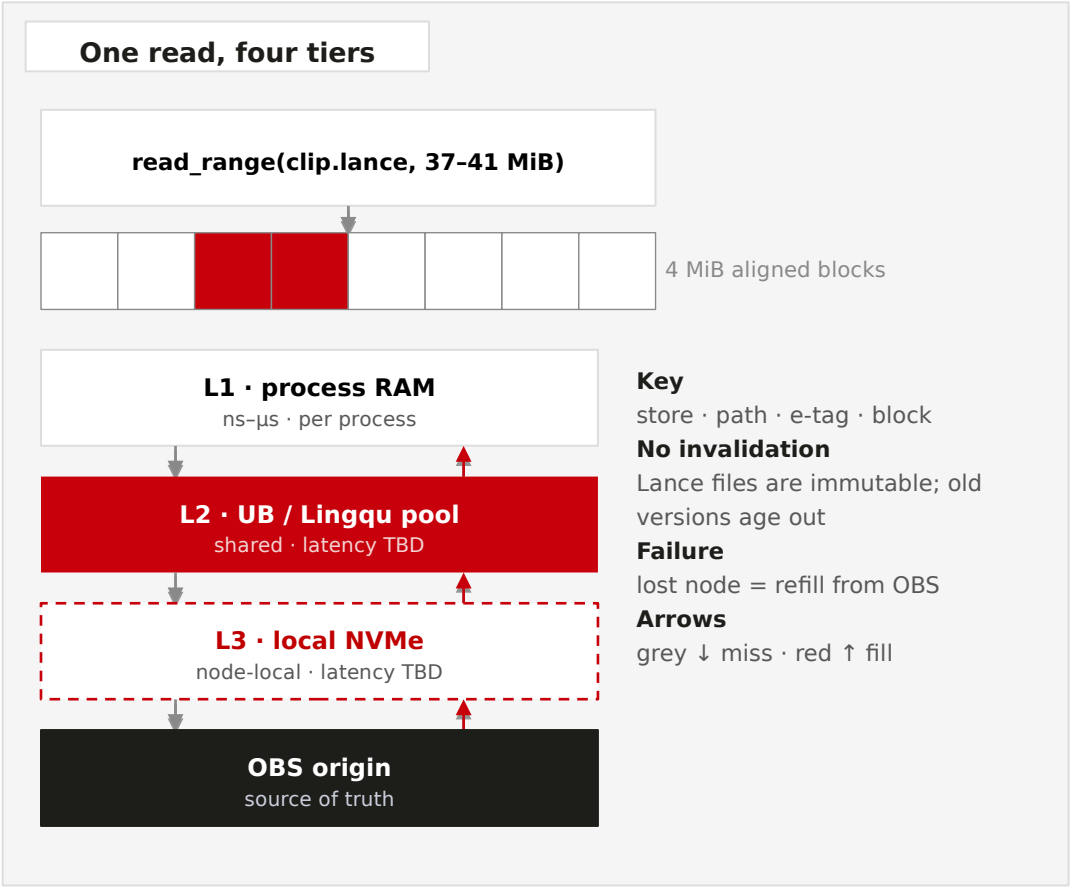
The gap for ADS: the biggest bytes, original objects, have no cache in Lance, and nothing is shared between processes, jobs or pipeline stages. Every read of a clip goes back to OBS.

Milliseconds come from reading only the needed range from a close tier; Lance has neither tier for object bytes today.

– Huawei Confidential –

KT1 · A tiered byte cache: RAM, UB pool, NVMe

[Proposal] read-through cache of immutable Lance file blocks under the reader · integration points exist in Lance [Source]



Lance integration points [Source]

Hook	Use in KT1
WrappingObjectStore	byte cache for data + index files
CacheTier::Local, get_without_promotion	index entries below RAM, scans skip promotion
CacheCodec (experimental)	serialize entries; decode error = miss
with_index_origin_latency(Low)	RQ reader stops paying for slow origin

Prior art [Reported]

System	Reported
Seneca (FAST 2026), cache for ML data loading	−45% makespan, 3.45× throughput
Lakehouse ANN, per-file cache over object store	rendezvous-hashed placement

First gate: p50 time-to-first-batch for a 4 GB clip served from L2 meets the ADS ms target (value set with ADS); OBS bytes per epoch fall.

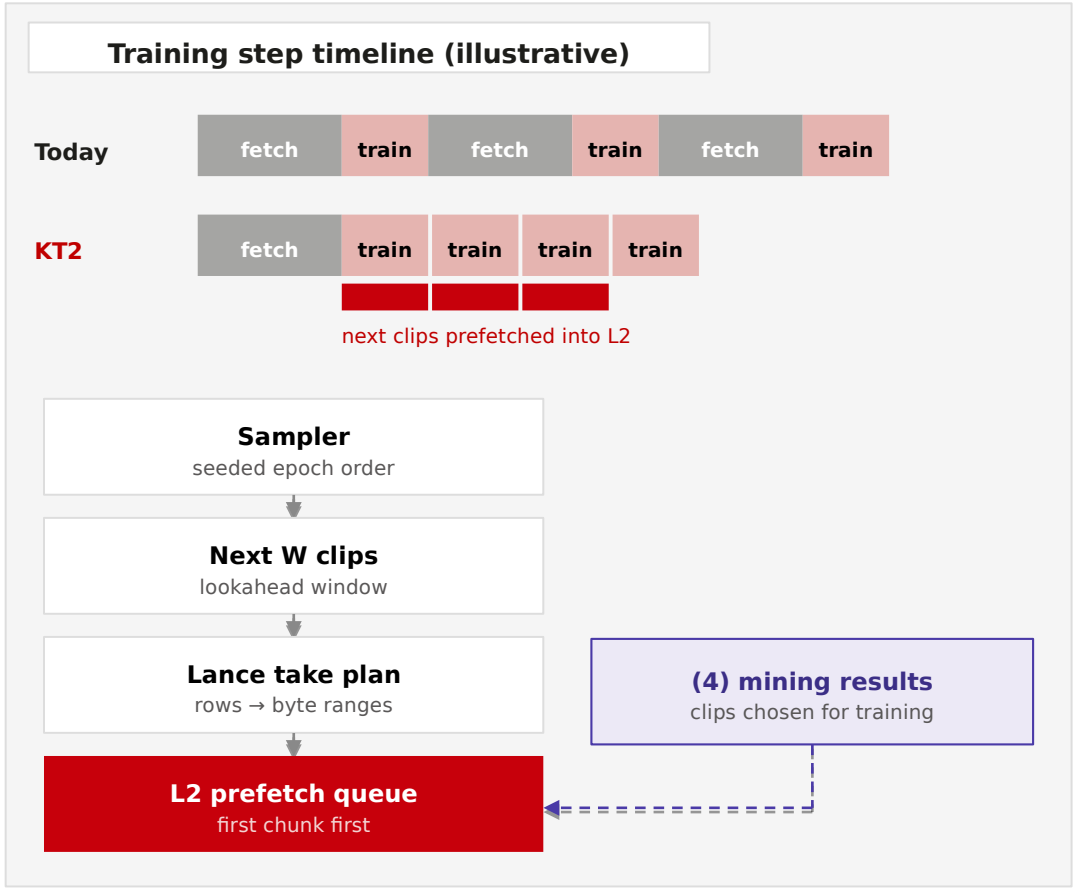
Immutable files make the cache simple: no invalidation, and any tier can be lost safely.

– Huawei Confidential –

HUAWEI

KT2 · Prefetch from the known training order

[Proposal] the sampler's order is fixed per epoch, so the next clips are known · timeline illustrative, not measured



Prefetch policy [Proposal]

- lookahead W clips, sized to the L2 budget per job
- first chunk of each clip first: time to first frame
- priority: training > mining hints > background warm-up
- cancel on epoch change or job stop
- one prefetch serves every job reading the same clip

Prior art (idea, numbers not transferred)

Work	Idea we reuse
Seneca (FAST 2026)	serve cached samples first across jobs
CoorDL (VLDB 2021)	coordinate data prep across jobs
SHADE (FAST 2023)	cache what matters most to training

First gate: GPU data-stall time per step near zero at the target batch rate, with prefetched bytes ≤ 1.2× bytes consumed.

The order is known before the step runs: fetch during compute, not after it.

– Huawei Confidential –

KT3 · One policy keeps valuable bytes close

[Proposal] utility per byte decides admission, eviction and tier · baselines are the S3-FIFO and TinyLFU backends Lance already has [Source]

Value per byte, one score for every object class

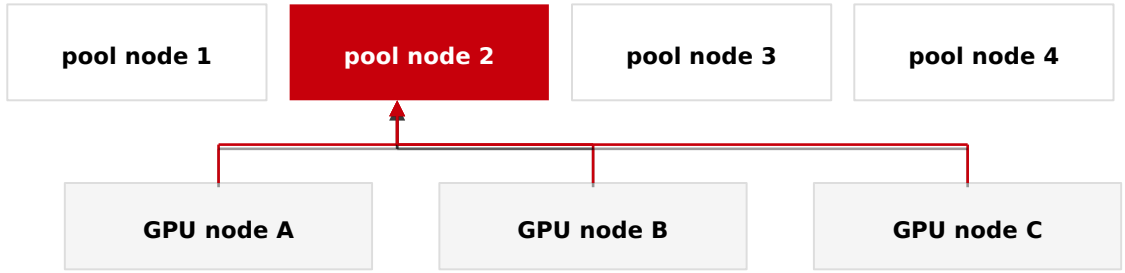
$$U(x) = \hat{F}(x) \cdot \hat{C}(x) \cdot \hat{R}(x) \div S(x)$$

$\hat{F}$ recent frequency · $\hat{C}$ load cost (tier, decode) · $\hat{R}$ reuse ahead (KT2 hints) · S bytes

Object	Size S	Load cost C	Reuse R
Clip byte range	large	OBS read	epochs, jobs
Decoded frame window	larger	decode (⑦ 1.5 s)	hot clips
IVF_RQ partition	medium	read + decode	queries
Tag allow-list	small	bitmap AND	repeat filters
Vector / tag columns	small	OBS read	mining

Encoded or decoded: cache the decoded window only when its reuse pays for its extra bytes; otherwise keep the clip encoded and decode again (links to ⑦ remote decoding).

Shared placement on the UB pool



clip 42 hashes to one owner (rendezvous); every job and stage reads that copy over UB

Compared with what Lance has

Policy	Status	Role
S3-FIFO (QuickCacheBackend)	default [Source]	baseline
TinyLFU (MokaCacheBackend)	built-in [Source]	baseline
OriginLatencyClass	RQ switch [Source]	becomes cost input $\hat{C}$

First gate: OBS bytes and refills per epoch fall vs S3-FIFO at equal pool size; no job slows by more than 5%.

One copy on the pool, chosen by value per byte, serves every job and stage.

– Huawei Confidential –

Summary · hooks exist in Lance, UB budget next

Three key technologies · gaps verified in Lance source · [Proposal] sizes S ≈ 1–2, M ≈ 3–5, L ≈ 6–10 engineer-weeks

KT1 · Tiered cache

64×

fewer bytes: window vs clip [Model]

· L2 UB pool + L3 NVMe
· no format change

KT2 · Prefetch

TBD

data-stall time per step

· training order known ahead
· (4) results as hints

KT3 · Policy

TBD

OBS bytes per epoch

· vs S3-FIFO and TinyLFU
· one copy for all jobs

ROADMAP · EACH PHASE HAS A GATE

1 · Trace + tier budget · S

UB / NVMe / OBS latency and bandwidth, ADS access trace

2 · KT1 tiers · M

L2 UB pool + L3 NVMe under Lance

3 · KT2 prefetch · M

sampler order + mining hints

4 · KT3 policy, upstream · L

cost-aware admission, Lance PRs

The origin stays the truth: every tier is a disposable copy of immutable files, so a miss, a lost node or a failed gate costs only speed, never correctness.

Lance has the hooks but no tier for object bytes; phase 1 measures UB and sets the ms target with ADS.

– Huawei Confidential –